Adult Income Classification Using Machine Learning

CP322 - Machine Learning

Dr. Sukhjit Singh Sehra

Submitted by: Touki Zahed and Adnan Awan

Date: 04/01/2026

Table of Contents

Title Page 1

Table of Contents 2

Abstract 3

Introduction 3

Project Description 4

Data Collection 5

Exploratory Data Analysis 6

Feature Selection 8

Methodology 9

Results and Experimental Analysis 12

Conclusion 16

References 17

Abstract

This project investigates the problem of income classification using the Adult Income dataset from the UCI Machine Learning Repository. The objective was to build machine learning models capable of predicting whether an individual earns more than $50,000 per year based on demographic and employment related attributes such as age, education, occupation, marital status, hours worked per week, and capital gain or loss. This problem is important because it represents a practical binary classification task with meaningful real world relevance in labor market and socioeconomic analysis.

The project followed a complete supervised machine learning pipeline. The dataset was cleaned by handling missing values represented by question marks, removing duplicate records, and separating the features into numerical and categorical groups. Numerical variables were imputed using the median and scaled, while categorical variables were imputed using the most frequent value and one hot encoded. The data were then split into stratified training and testing sets to preserve class balance. A Dummy Classifier was used as a baseline, followed by Logistic Regression, Linear Support Vector Machine, and Random Forest. Hyperparameter tuning was performed using GridSearchCV for the Random Forest model.

The tuned Random Forest achieved the strongest overall performance, with a test accuracy of 86.97% and a ROC-AUC of approximately 0.922. These results exceeded the project requirement that the best chosen model achieve at least 80% accuracy. The study also showed that the untuned Random Forest exhibited substantial overfitting, while hyperparameter tuning improved generalization. Overall, the project demonstrates that careful preprocessing, appropriate model selection, and hyperparameter tuning can produce strong and interpretable results for income classification.

Introduction

Machine learning classification methods are widely used to support decision making in settings where the goal is to assign observations to one of several predefined categories. Income

classification is a common example of this type of problem because it involves predicting whether a person belongs to a lower income or higher income group based on attributes such as education, occupation, age, and work related characteristics. In practice, this type of task can be relevant to labor market research, policy analysis, and studies of economic inequality.

The goal of this project was to use machine learning techniques to classify whether an individual earns more than $50,000 per year using the Adult Income dataset. The project was designed to follow a full machine learning workflow, beginning with data cleaning and preprocessing, continuing through model training and tuning, and ending with evaluation and comparison of multiple models. Since the assignment specifically required the use of several models, preprocessing, evaluation, and hyperparameter tuning, the project focused on building a reproducible pipeline and comparing model behavior rather than only reporting a single final score.

A second objective of the project was to understand how preprocessing choices and model complexity affect performance. This included examining missing values, class imbalance, scaling requirements, and potential overfitting. In addition to achieving a strong predictive result, the report aims to explain why certain methods performed better than others and what limitations remained in the final solution.

Project Description

This project addresses a binary classification problem in which the target variable is **income**, with two classes that include **<=50K** and **>50K**. The central question is whether machine learning models can accurately predict a person's income category using demographic and employment attributes.

From a real world perspective, this problem is more meaningful than simply predicting abstract labels. Variables such as education level, occupation, marital status, capital gain, and working hours can reflect broader structural patterns in labor markets and access to higher

paying opportunities. For that reason, the project does not only report predictive accuracy, but also interprets the findings in the context of the underlying socioeconomic problem.

The project was implemented in Python using a modular structure. Separate classes were used for data loading, preprocessing, model training, and evaluation, and a `main.py` file coordinated the full workflow. This design improved reproducibility and allowed the final results to be regenerated consistently.

Data Collection

The dataset used in this project is the **Adult Income dataset**, also known as the Census Income dataset, obtained from the **UCI Machine Learning Repository**. It contains demographic, educational, and employment related information for individuals and is commonly used as a benchmark dataset for classification tasks.

The original dataset loaded in the project contained **48,842 rows** and **15 columns**. After cleaning, including duplicate removal, the working dataset contained **48,790 rows** and **15 columns**. The target variable was **income**, and the remaining variables served as predictors. The predictors included both numerical and categorical features. Numerical variables included age, final weight, education number, capital gain, capital loss, and hours per week. Categorical variables included workclass, education, marital status, occupation, relationship, race, sex, and native country.

The dataset is suitable for this assignment for several reasons. First, it contains enough observations to train and test multiple machine learning models. Second, it includes both numeric and categorical data, which makes preprocessing more realistic and meaningful. Third, the problem is interpretable and allows for a useful discussion of relationships between social and economic variables.

Table 1. Dataset Summary

Item	Value
Raw Observations	48,842
Cleaned Observations	48,790
Total Columns	15
Predictor Columns	14
Target column	income
Numerical Predictors	6
Categorical Predictors	8
Training Instances	39,032
Testing Instances	9,758

In the final experimental setup, the cleaned dataset was split into **39,032 training instances** and **9,758 testing instances**, using an 80/20 stratified train test split.

Exploratory Data Analysis

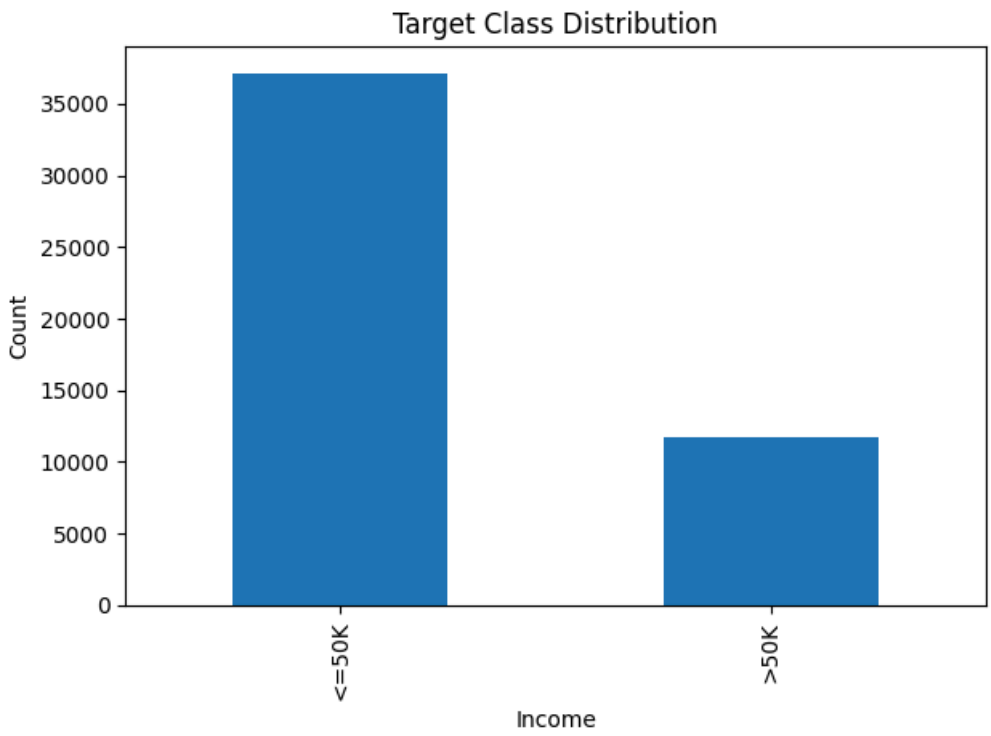
Exploratory Data Analysis was performed to understand the structure of the data and identify issues that could affect model performance. Several aspects were particularly important for this project, including missing values, class imbalance, scaling requirements, skewed variables, and relationships between features and the target.

One important observation was the presence of missing values represented as question marks in several categorical columns, especially **workclass**, **occupation**, and **native_country**. These values were not stored as standard null values, so part of the cleaning process involved

stripping whitespace and converting question marks into missing values. This step was important because leaving them untreated would cause the models to interpret question marks as ordinary categories rather than missing information.

The target distribution showed a noticeable class imbalance. The majority class was the **<=50K** group, which accounted for about 76% of the observations, while the **>50K** group formed the minority class. This imbalance was also visible in the performance of the baseline Dummy Classifier, which achieved about 76.06% accuracy by always predicting the majority class. This result confirmed that a model needed to perform well beyond the baseline to be considered meaningful.

Figure 1. Distribution of income classes in the cleaned dataset



The numerical variables also revealed useful patterns. Variables such as **capital_gain** and **capital_loss** were heavily skewed, with many observations near zero and a smaller number of larger values. In contrast, variables such as **age** and **hours_per_week** were easier to interpret

directly. Since some of the selected models, especially Logistic Regression and Linear SVM, are sensitive to feature scale, scaling of numerical features was necessary.

EDA also suggested that several variables were likely to be informative for classification. In general, higher values of **education_num**, nonzero **capital_gain**, and longer **hours_per_week** appeared to be associated with higher income. Categorical variables such as **marital_status**, **occupation**, and **relationship** also appeared likely to contain useful predictive structure. These relationships are reasonable from a socioeconomic perspective, since higher educational attainment, full time or overtime employment, and certain occupational categories are commonly associated with higher earnings.

Outlier handling was considered as part of the analysis. However, no rows were removed solely as outliers. This decision was based on the fact that extreme values in variables such as capital gain and work hours may represent real economic behavior rather than data entry errors. Tree based models are generally robust to such values, and scaling plus imputation was considered sufficient for the linear models used in this project.

Feature Selection

Feature selection in this project was conservative. Rather than aggressively removing variables, the project retained all original predictors after cleaning and relied on preprocessing, model comparison, and evaluation to determine which modeling approach made the best use of the available information.

This choice was appropriate for several reasons. First, the dataset contains a manageable number of features, so dimensionality was not so large that it created a major computational burden after encoding. Second, variables that may appear weak on their own can still contribute useful information when combined with others. Third, the use of both linear and tree based models made it valuable to preserve a common feature set across models so that the comparison would remain fair.

Categorical variables were transformed through one hot encoding, which expanded the feature space but allowed the algorithms to represent nonnumeric information properly. Numerical variables were standardized when appropriate. No manual elimination of predictors was performed before model training. Instead, model performance and interpretability were examined after fitting.

Although extensive handcrafted feature engineering was not the main focus of the final comparison, the project considered the possibility of engineered variables as future work. For example, relationships involving work intensity, net capital, or grouped age categories could potentially improve performance further. However, the main goal of the current project was to demonstrate a robust and reproducible baseline pipeline with meaningful model comparison and tuning.

Methodology

Data Cleaning and Preparation:

The first step in the pipeline was to load the Adult Income data and clean it so that it could be used for machine learning. The cleaning process included stripping extra whitespace from string columns, converting question marks to missing values, and removing duplicate rows. The target column was then separated from the predictor variables.

Handling missing values was an important methodological decision. Rather than deleting all incomplete observations, the project used imputation within a preprocessing pipeline. Numerical variables were imputed using the median, while categorical variables were imputed using the most frequent category. This approach allowed the project to preserve most of the data while avoiding leakage from the test set.

Train Test Split and Preprocessing Pipeline:

After cleaning, the dataset was split into training and testing sets using an 80/20 split. Stratification was applied with respect to the target variable so that the class distribution would remain similar in both subsets. This was important because the dataset is moderately imbalanced.

A column based preprocessing pipeline was then created. Numerical features were passed through a pipeline consisting of median imputation followed by standardization. Categorical features were passed through a pipeline consisting of most frequent imputation followed by one hot encoding with unknown categories ignored. The full preprocessing object was built using a `ColumnTransformer`, which ensured that the same transformations would be consistently applied during training and evaluation.

Models:

The project compared four classification models. The first model was a **Dummy Classifier**, which always predicts the majority class. This model served as a baseline and helped determine whether the more advanced models actually learned useful patterns beyond class imbalance.

The second model was **Logistic Regression**, a linear classification model that is interpretable, efficient, and often performs well on structured tabular data after appropriate preprocessing.

The third model was **Linear Support Vector Machine**, implemented using `LinearSVC`. This model is also well suited to high dimensional feature spaces created by one hot encoding and can provide strong classification performance when the classes are separable in a transformed feature space.

The fourth model was **Random Forest**, an ensemble method that combines many decision trees and is capable of modeling nonlinear relationships and interactions among variables.

Hyperparameter Tuning:

To improve performance and reduce overfitting, hyperparameter tuning was performed for the Random Forest model using **GridSearchCV**. The search space included:

- number of trees: 100 and 200
- maximum depth: None, 10, and 20
- minimum samples split: 2 and 5
- minimum samples leaf: 1 and 2

Three fold cross validation was used inside the grid search, and accuracy served as the optimization metric. The best parameter combination selected by the tuning process was:

- n_estimators = 200
- max_depth = None
- min_samples_split = 2
- min_samples_leaf = 2

Evaluation Metrics:

Model performance was evaluated on the held out test set using several metrics;

- Accuracy
- Weighted Precision
- Weighted Recall
- Weighted F1 score
- ROC-AUC
- Confusion Matrix

Accuracy was the main headline metric because the assignment requires the best chosen model to achieve at least 80% accuracy. However, additional metrics were important because the classes are imbalanced and accuracy alone can be misleading. ROC-AUC provided a threshold independent measure of separability, while the weighted metrics summarized performance across both classes.

Results and Experimental Analysis

Model Comparison:

Model	**Train Accuracy**	**Test Accuracy**	**Weighted Precision**	**Weighted Recall**	**Weighted F1**	**ROC-AUC**
DummyClassifier	0.7606	0.7606	0.5785	0.7606	0.6572	0.5000
LogisticRegression	0.8527	0.8556	0.8498	0.8556	0.8510	0.9098
LinearSVM	0.8528	0.8565	0.8505	0.8565	0.8514	0.9094
RandomForest	0.9999	0.8570	0.8522	0.8570	0.8536	0.9082
RandomForest_Tuned	0.9034	0.8697	0.8648	0.8697	0.8650	0.9222

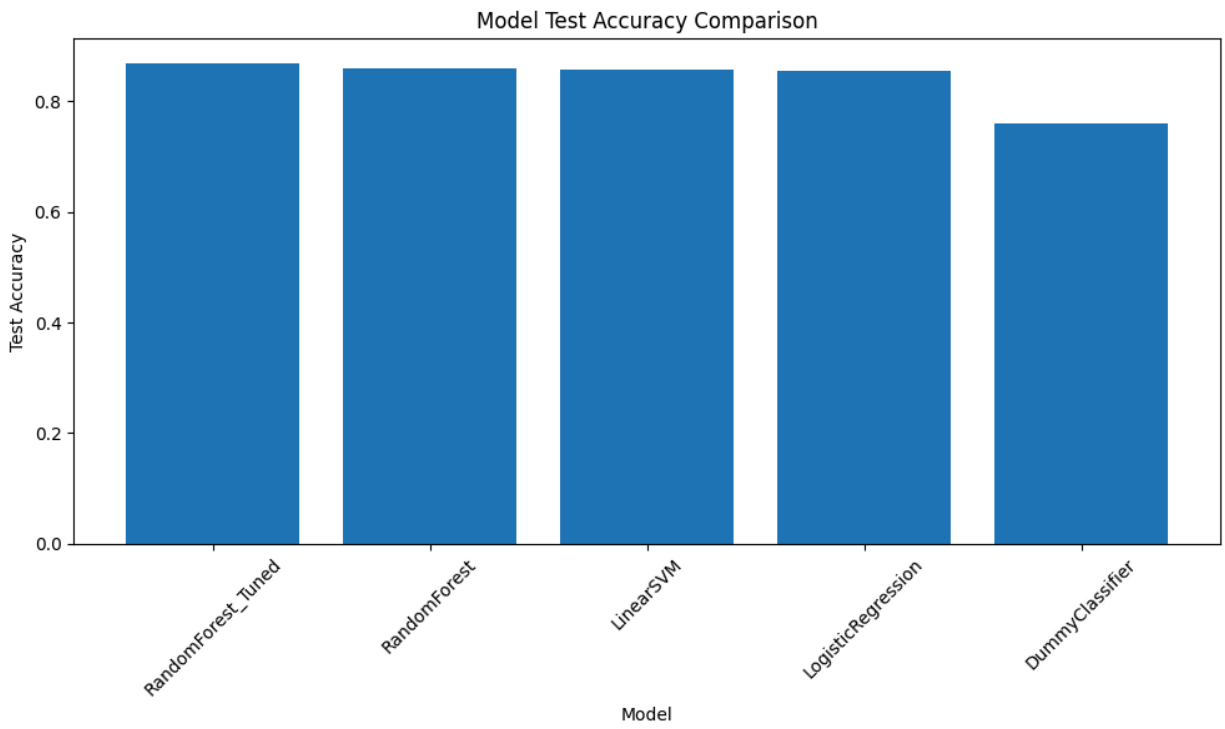
The Dummy Classifier established a baseline accuracy of about 76.06%, which closely matched the proportion of the majority class. This confirmed that the dataset is imbalanced and that a model needs to perform substantially better than 76% to demonstrate meaningful learning.

Logistic Regression and Linear SVM both produced strong results, with test accuracies above 85.5% and ROC-AUC values around 0.91. These results showed that the preprocessing pipeline was effective and that the relationship between the predictors and target can be captured reasonably well by linear decision boundaries in the transformed feature space.

The untuned Random Forest slightly outperformed the linear models on test accuracy, but its train accuracy was nearly 100%. This extremely high training performance combined with a noticeably lower test accuracy is a clear sign of overfitting. The model fit the training data too closely and did not generalize as well as expected.

The tuned Random Forest achieved the best balance between fit and generalization. Its training accuracy decreased to about 90.34%, while its test accuracy increased to 86.97%. This result indicates that hyperparameter tuning reduced overfitting and improved out of sample performance. The tuned Random Forest also achieved the highest weighted F1 score and the highest ROC-AUC, making it the strongest overall model in the study.

Figure 2. Test accuracy comparison across models



Best Model Analysis

The selected final model was the **Tuned Random Forest**. It achieved the highest test accuracy, the highest ROC-AUC, and the strongest balance across the weighted evaluation metrics. Most importantly, it exceeded the assignment requirement that the best model achieve at least 80% accuracy.

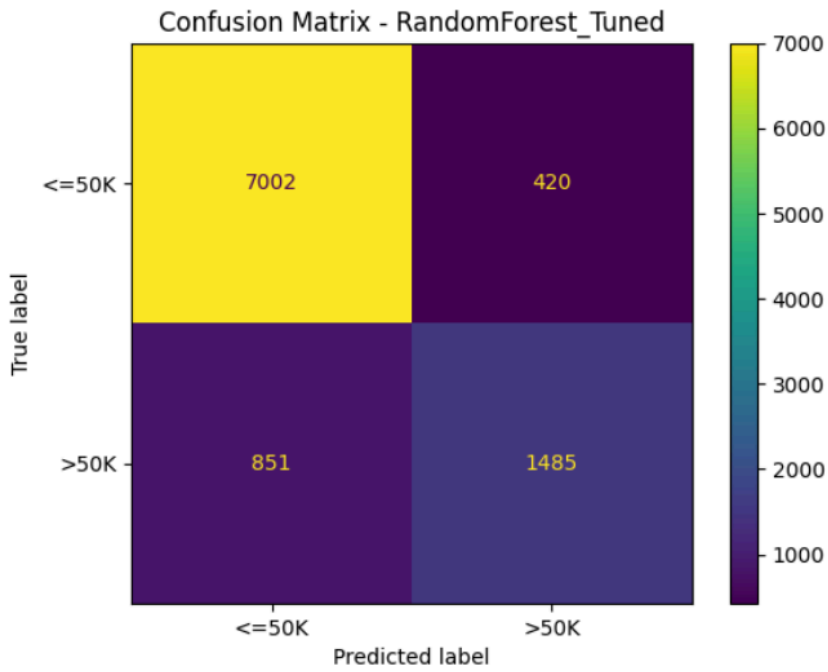
The tuned Random Forest was effective for several reasons. First, it can naturally handle nonlinear relationships and interactions among variables, which are likely present in a dataset containing education, capital gain, marital status, and occupation. Second, it is less sensitive to

monotonic scaling than linear models, which makes it robust when numerical variables are skewed. Third, the ensemble structure helps average out individual tree instability.

These results can be interpreted in a broader real-world context. The model's strong performance suggests that income level is not determined by a single factor, but by the interaction of multiple socioeconomic characteristics. For example, higher levels of education often provide access to better-paying occupations, while longer working hours may reflect full-time or overtime employment associated with higher income. Similarly, nonzero capital gain may indicate access to investments or accumulated wealth, which further increases earning potential. The fact that the Random Forest model performed best supports the idea that these relationships are not purely linear, but instead involve complex interactions between education, occupation, work patterns, and financial factors. This aligns with real-world labor market dynamics, where income is shaped by a combination of demographic, economic, and structural influences rather than any single variable.

The confusion matrix for the tuned Random Forest showed strong performance for the majority class and improved performance for the minority class compared with the baseline model. Even so, the higher income class remained somewhat harder to predict, which is expected because the dataset is imbalanced and the characteristics of the two classes overlap.

Figure 3. Confusion matrix for the tuned Random Forest



Discussion of Overfitting, Generalization, and Limitations

One of the most important findings in the project was the difference between the untuned and tuned Random Forest models. The untuned model achieved almost perfect training accuracy, which is usually a warning sign in tabular classification problems of this size. After tuning, the Random Forest still performed strongly but no longer memorized the training data to the same extent. This improved generalization and demonstrates why hyperparameter tuning is necessary rather than optional.

The linear models also produced competitive performance, which is significant because they are simpler, faster, and easier to interpret. This suggests that the preprocessing pipeline, especially one hot encoding and scaling, made the dataset well suited to linear classification methods. In some real world settings, a slightly less accurate but more interpretable model such as Logistic Regression might still be preferred. However, for this project, the tuned Random Forest was selected because predictive performance was the primary criterion.

There are also limitations to acknowledge. First, the project did not use extensive manual feature engineering or feature selection beyond preprocessing. Second, the dataset reflects a specific historical and demographic context, so model behavior should not be interpreted as universally applicable. Third, performance could potentially be improved further with additional models such as Gradient Boosting or XGBoost, or by incorporating threshold tuning and class weighting.

Conclusion

This project successfully applied machine learning methods to the Adult Income classification problem. Beginning with a raw tabular dataset containing both categorical and numerical variables, the project developed a complete workflow that included data cleaning, missing value handling, encoding, scaling, train test splitting, model comparison, evaluation, and hyperparameter tuning.

Among the tested models, the tuned Random Forest achieved the best overall performance, with a test accuracy of 86.97% and a ROC-AUC of approximately 0.922. This result exceeded the 80% threshold required by the assignment and demonstrated that careful tuning improved generalization beyond the untuned Random Forest. Logistic Regression and Linear SVM also performed strongly, showing that well preprocessed tabular data can often be modeled effectively with relatively simple methods. The project also highlighted several important machine learning lessons. Data preprocessing has a direct impact on model quality. Baselines are necessary for meaningful comparison. Overfitting must be evaluated by comparing training and test performance rather than only looking at a single accuracy number. Finally, hyperparameter tuning can substantially improve model behavior even when the base model is already strong. Future work could include more advanced feature engineering, additional ensemble methods, threshold optimization for the minority class, and deeper analysis of fairness and interpretability. Even in its current form, however, the project demonstrates a complete and well organized machine learning pipeline that produces strong, reproducible results for a real classification task.

References

Becker, B., & Kohavi, R. (1996). *Adult* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5XW20

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*, 273–297. https://doi.org/10.1007/BF00994018

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., & Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html